AIRPORT OPERATIONS & FLIGHT MANAGEMENT SYSTEM

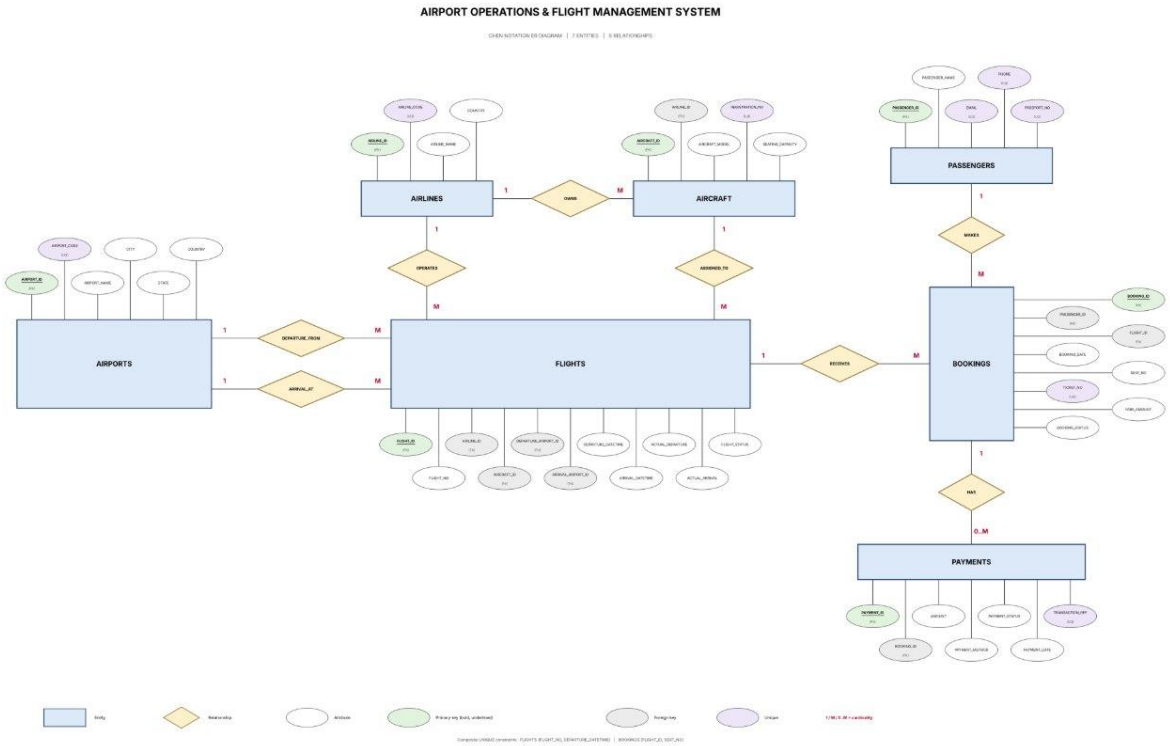
END-TO-END MYSQL PROJECT

ER Diagram • Database Schema • Sample Data • Indexes • Views • Stored Procedures • Triggers • Business Rules •

Database: AIRPORT_MANAGEMENT
Project Type: Real-world relational database implementation

TABLE OF CONTENTS

Entity-Relationship (ER) Diagram



Project Overview

- PART 1: DATABASE AND TABLE CREATION
- PART 2: SAMPLE DATA
- PART 3: INDEXES
- PART 4: VIEWS
- PART 5: STORED PROCEDURES
- PART 6: TRIGGERS
- PART 7: QUERY BANK – LEVEL 1: SIMPLE QUERIES (Q1–Q6)
- PART 8: QUERY BANK – LEVEL 2: INTERMEDIATE QUERIES (Q7–Q14)

- PART 9: QUERY BANK – LEVEL 3: ADVANCED QUERIES (Q15–Q23)
- PART 10: QUERY BANK – LEVEL 4: COMPLEX QUERIES (Q24–Q33)
- PART 11: QUERY BANK – LEVEL 5: REAL-WORLD AND ADVANCED SQL (Q34–Q43)
- PART 12: QUERY BANK – LEVEL 6: DATABASE PROGRAMMING (Q44–Q47)
- PART 13: DEMO CALLS – VIEWS / STORED PROCEDURES / TRIGGERS
- PART 14: TESTING AND VALIDATION
- PART 15: PROJECT CONCLUSION

1.1 Entity Summary

Entity	Entity Type	Primary Key	Description
AIRPORTS	Strong	AIRPORT_ID	Stores airport details and locations.
AIRLINES	Strong	AIRLINE_ID	Stores airline information.
AIRCRAFT	Strong	AIRCRAFT_ID	Stores aircraft details owned by airlines.
FLIGHTS	Strong	FLIGHT_ID	Represents an individual dated flight journey.
PASSENGERS	Strong	PASSENGER_ID	Stores passenger information.
BOOKINGS	Strong	BOOKING_ID	Represents a passenger's reservation for a flight.
PAYMENTS	Strong	PAYMENT_ID	Stores payment attempts and transaction details.

1.2 Entity Relationships

No.	Entity 1	Relationship	Entity 2	Cardinality
1	AIRLINES	OWNS	AIRCRAFT	1:M
2	AIRLINES	OPERATES	FLIGHTS	1:M
3	AIRCRAFT	ASSIGNED_TO	FLIGHTS	1:M

4	AIRPORTS	DEPARTURE_FROM	FLIGHTS	1:M
5	AIRPORTS	ARRIVAL_AT	FLIGHTS	1:M
6	PASSENGERS	MAKES	BOOKINGS	1:M
7	FLIGHTS	RECEIVES	BOOKINGS	1:M
8	BOOKINGS	HAS	PAYMENTS	1:0..M

1.3 Relationship Explanation

AIRLINES – AIRCRAFT

One airline can own multiple aircraft, while each aircraft belongs to one airline.

AIRLINES – FLIGHTS

One airline can operate multiple flights, while each flight is operated by one airline.

AIRCRAFT – FLIGHTS

One aircraft can be assigned to multiple flights over time, while each flight uses one aircraft.

AIRPORTS – FLIGHTS (Departure)

One airport can be the departure airport for multiple flights. Each flight has one departure airport.

AIRPORTS – FLIGHTS (Arrival)

One airport can be the arrival airport for multiple flights. Each flight has one arrival airport.

PASSENGERS – BOOKINGS

One passenger can make multiple bookings, while each booking belongs to one passenger.

FLIGHTS – BOOKINGS

One flight can receive multiple bookings, while each booking is associated with one flight.

BOOKINGS – PAYMENTS

One booking can have zero or multiple payment attempts, while each payment belongs to one booking.

1.4 Database Design Considerations

1. Each FLIGHTS record represents one actual dated flight journey.

2. The same flight number can appear on different dates.
3. Departure and arrival airports are maintained as separate foreign keys.
4. Passenger details are maintained independently of bookings.
5. Seat and ticket information are maintained in BOOKINGS.
6. Multiple payment attempts are allowed for one booking.
7. A booking may exist without a successful payment.
8. The database must prevent duplicate seat assignments on the same flight.

2.1 Project Description

The Airport Operations & Flight Management System is a relational database application designed to manage airport-related information and flight reservation activities.

The system maintains airport locations, airline details, aircraft registrations, flight schedules, passenger information, bookings and payment transactions.

It supports different operational scenarios, including flight searching, passenger reservations, seat allocation, payment processing and revenue reporting.

The project demonstrates the application of MySQL database concepts through a practical business scenario.

2.2 Project Objectives

- Maintain airport and airline records.
- Manage aircraft and flight information.
- Maintain passenger information.
- Handle flight reservations and seat allocation.
- Record payment transactions.
- Generate operational and financial reports.
- Enforce database constraints and business rules.
- Improve query performance using indexes.
- Automate operations using stored procedures and triggers.

PART 1: DATABASE AND TABLE CREATION

1.1 Database Creation

Database name: airport_db

The database stores all information related to airport operations, flights, passengers, bookings and payments.

Concepts:

- CREATE DATABASE
- DROP DATABASE
- USE
- SHOW DATABASES
- SELECT DATABASE()

Implementation section:

CREATE DATABASE AIRPORT_MANAGEMENT;

1.2 Table Creation Order

The tables must be created according to their foreign-key dependencies.

Order	Table	Foreign-key dependency
1	AIRPORTS	Independent
2	AIRLINES	Independent
3	PASSENGERS	Independent
4	AIRCRAFT	AIRLINES
5	FLIGHTS	AIRLINES, AIRCRAFT, AIRPORTS
6	BOOKINGS	PASSENGERS, FLIGHTS
7	PAYMENTS	BOOKINGS

1.3 Table Structures

TABLE 1 – AIRPORTS

Purpose: Stores airport identification and location information.

Column	Data Type	Constraints

AIRPORT_ID	INT	PRIMARY KEY, AUTO_INCREMENT
AIRPORT_CODE	CHAR(3)	NOT NULL, UNIQUE
AIRPORT_NAME	VARCHAR(100)	NOT NULL
CITY	VARCHAR(50)	NOT NULL
STATE	VARCHAR(50)	NOT NULL
COUNTRY	VARCHAR(50)	NOT NULL

TABLE 2 – AIRLINES

Purpose: Stores information about airlines operating flights.

Column	Data Type	Constraints
AIRLINE_ID	INT	PRIMARY KEY, AUTO_INCREMENT
AIRLINE_CODE	VARCHAR(5)	NOT NULL, UNIQUE
AIRLINE_NAME	VARCHAR(100)	NOT NULL
COUNTRY	VARCHAR(50)	NOT NULL

TABLE 3 – PASSENGERS

Purpose: Stores passenger identification and contact information.

Column	Data Type	Constraints
PASSENGER_ID	INT	PRIMARY KEY, AUTO_INCREMENT
PASSENGER_NAME	VARCHAR(100)	NOT NULL
EMAIL	VARCHAR(100)	UNIQUE
PHONE	VARCHAR(15)	UNIQUE
PASSPORT_NO	VARCHAR(20)	UNIQUE

TABLE 4 – AIRCRAFT

Purpose: Stores aircraft registration, model and airline ownership details.

Column	Data Type	Constraints
AIRCRAFT_ID	INT	PRIMARY KEY, AUTO_INCREMENT
AIRLINE_ID	INT	FOREIGN KEY, NOT NULL

AIRCRAFT_MODEL	VARCHAR(100)	NOT NULL
REGISTRATION_NO	VARCHAR(20)	NOT NULL, UNIQUE
SEATING_CAPACITY	SMALLINT	NOT NULL, CHECK (SEATING_CAPACITY > 0)

Foreign key relationship:

AIRLINE_ID references AIRLINES(AIRLINE_ID).

TABLE 5 – FLIGHTS

Purpose: Stores scheduled and actual flight journey information.

Column	Data Type	Constraints
FLIGHT_ID	INT	PRIMARY KEY, AUTO_INCREMENT
FLIGHT_NO	VARCHAR(10)	NOT NULL
AIRLINE_ID	INT	FOREIGN KEY, NOT NULL
AIRCRAFT_ID	INT	FOREIGN KEY, NOT NULL
DEPARTURE_AIRPORT_ID	INT	FOREIGN KEY, NOT NULL
ARRIVAL_AIRPORT_ID	INT	FOREIGN KEY, NOT NULL
DEPARTURE_DATETIME	DATETIME	NOT NULL
ARRIVAL_DATETIME	DATETIME	NOT NULL
ACTUAL_DEPARTURE	DATETIME	NULL
ACTUAL_ARRIVAL	DATETIME	NULL
FLIGHT_STATUS	VARCHAR(20)	NOT NULL

Foreign key relationships:

- AIRLINE_ID → AIRLINES(AIRLINE_ID)
- AIRCRAFT_ID → AIRCRAFT(AIRCRAFT_ID)
- DEPARTURE_AIRPORT_ID → AIRPORTS(AIRPORT_ID)
- ARRIVAL_AIRPORT_ID → AIRPORTS(AIRPORT_ID)

Additional constraints:

- UNIQUE (FLIGHT_NO, DEPARTURE_DATETIME)

- CHECK (DEPARTURE_AIRPORT_ID

ARRIVAL_AIRPORT_ID)

- CHECK (ARRIVAL_DATETIME > DEPARTURE_DATETIME)

TABLE 6 – BOOKINGS

Purpose: Stores passenger flight reservations, seat assignments and ticket information.

Column	Data Type	Constraints
BOOKING_ID	INT	PRIMARY KEY, AUTO_INCREMENT
PASSENGER_ID	INT	FOREIGN KEY, NOT NULL
FLIGHT_ID	INT	FOREIGN KEY, NOT NULL
BOOKING_DATE	DATETIME	NOT NULL
SEAT_NO	VARCHAR(5)	NULL
TICKET_NO	VARCHAR(30)	UNIQUE, NULL
FARE_AMOUNT	DECIMAL(10,2)	NOT NULL, CHECK (FARE_AMOUNT >= 0)
BOOKING_STATUS	VARCHAR(20)	NOT NULL

Foreign key relationships:

- PASSENGER_ID → PASSENGERS(PASSENGER_ID)
- FLIGHT_ID → FLIGHTS(FLIGHT_ID)

Additional constraint:

- UNIQUE (FLIGHT_ID, SEAT_NO)

TABLE 7 – PAYMENTS

Purpose: Stores payment attempts and transaction information associated with bookings.

Column	Data Type	Constraints
PAYMENT_ID	INT	PRIMARY KEY, AUTO_INCREMENT
BOOKING_ID	INT	FOREIGN KEY, NOT NULL
AMOUNT	DECIMAL(10,2)	NOT NULL, CHECK (AMOUNT > 0)

PAYMENT_METHOD	VARCHAR(20)	NOT NULL
PAYMENT_STATUS	VARCHAR(20)	NOT NULL
PAYMENT_DATE	DATETIME	NOT NULL
TRANSACTION_REF	VARCHAR(50)	UNIQUE, NULL

Foreign key relationship:

BOOKING_ID references BOOKINGS(BOOKING_ID).

Design note: BOOKING_ID is not UNIQUE in PAYMENTS because one booking can have multiple payment attempts.

1.4 Primary Keys and Foreign Keys

Topics:

- Primary key identification
- Foreign key references
- One-to-many relationships
- Referential integrity
- Composite unique constraints
- Candidate keys
- Surrogate keys

1.5 Table Modification

Topics:

- ALTER TABLE ADD
- ALTER TABLE MODIFY
- ALTER TABLE DROP COLUMN
- RENAME TABLE
- DROP TABLE
- DESC
- SHOW CREATE TABLE

1.6 Database Verification

Topics:

- SHOW DATABASES

- SHOW TABLES
- DESC table_name
- SELECT *
- SHOW CREATE TABLE
- Foreign-key verification

PART 2: SAMPLE DATA

```
CREATE TABLE AIRPORTS (
   AIRPORT_ID INT PRIMARY KEY AUTO_INCREMENT,
   AIRPORT_CODE CHAR(3) NOT NULL UNIQUE,
   AIRPORT_NAME VARCHAR(100) NOT NULL,
   CITY VARCHAR(50) NOT NULL,
   STATE VARCHAR(50) NOT NULL,
   COUNTRY VARCHAR(50) NOT NULL
);

CREATE TABLE AIRLINES (
   AIRLINE_ID INT PRIMARY KEY AUTO_INCREMENT,
   AIRLINE_CODE VARCHAR(5) NOT NULL UNIQUE,
   AIRLINE_NAME VARCHAR(100) NOT NULL,
   COUNTRY VARCHAR(50) NOT NULL
);

CREATE TABLE AIRCRAFT (
   AIRCRAFT_ID INT PRIMARY KEY AUTO_INCREMENT,
   AIRLINE_ID INT NOT NULL,
   AIRCRAFT_MODEL VARCHAR(100) NOT NULL,
   REGISTRATION_NO VARCHAR(20) NOT NULL UNIQUE,
   SEATING_CAPACITY SMALLINT NOT NULL CHECK (SEATING_CAPACITY >
0),
```

```
    FOREIGN KEY (AIRLINE_ID) REFERENCES AIRLINES(AIRLINE_ID)
);

CREATE TABLE FLIGHTS (
    FLIGHT_ID INT PRIMARY KEY AUTO_INCREMENT,
    FLIGHT_NO VARCHAR(10) NOT NULL,
    AIRLINE_ID INT NOT NULL,
    AIRCRAFT_ID INT NOT NULL,
    DEPARTURE_AIRPORT_ID INT NOT NULL,
    ARRIVAL_AIRPORT_ID INT NOT NULL,
    DEPARTURE_DATETIME DATETIME NOT NULL,
    ARRIVAL_DATETIME DATETIME NOT NULL,
    ACTUAL_DEPARTURE DATETIME,
    ACTUAL_ARRIVAL DATETIME,
    FLIGHT_STATUS VARCHAR(20) NOT NULL,

    UNIQUE (FLIGHT_NO, DEPARTURE_DATETIME),

    CHECK (DEPARTURE_AIRPORT_ID <> ARRIVAL_AIRPORT_ID),
    CHECK (ARRIVAL_DATETIME > DEPARTURE_DATETIME),

    FOREIGN KEY (AIRLINE_ID) REFERENCES AIRLINES(AIRLINE_ID),
    FOREIGN KEY (AIRCRAFT_ID) REFERENCES AIRCRAFT(AIRCRAFT_ID),
    FOREIGN KEY (DEPARTURE_AIRPORT_ID) REFERENCES
AIRPORTS(AIRPORT_ID),
    FOREIGN KEY (ARRIVAL_AIRPORT_ID) REFERENCES
AIRPORTS(AIRPORT_ID)
);

CREATE TABLE PASSENGERS (
```

```
    PASSENGER_ID INT PRIMARY KEY AUTO_INCREMENT,
    PASSENGER_NAME VARCHAR(100) NOT NULL,
    EMAIL VARCHAR(100) UNIQUE,
    PHONE VARCHAR(15) UNIQUE,
    PASSPORT_NO VARCHAR(20) UNIQUE
);

CREATE TABLE BOOKINGS (
    BOOKING_ID INT PRIMARY KEY AUTO_INCREMENT,
    PASSENGER_ID INT NOT NULL,
    FLIGHT_ID INT NOT NULL,
    BOOKING_DATE DATETIME NOT NULL,
    SEAT_NO VARCHAR(5),
    TICKET_NO VARCHAR(30) UNIQUE,
    FARE_AMOUNT DECIMAL(10,2) NOT NULL CHECK (FARE_AMOUNT >= 0),
    BOOKING_STATUS VARCHAR(20) NOT NULL,

    UNIQUE (FLIGHT_ID, SEAT_NO),

    FOREIGN KEY (PASSENGER_ID) REFERENCES
PASSENGERS(PASSENGER_ID),
    FOREIGN KEY (FLIGHT_ID) REFERENCES FLIGHTS(FLIGHT_ID)
);

CREATE TABLE PAYMENTS (
    PAYMENT_ID INT PRIMARY KEY AUTO_INCREMENT,
    BOOKING_ID INT NOT NULL,
    AMOUNT DECIMAL(10,2) NOT NULL CHECK (AMOUNT > 0),
    PAYMENT_METHOD VARCHAR(20) NOT NULL,
```

```
    PAYMENT_STATUS VARCHAR(20) NOT NULL,
    PAYMENT_DATE DATETIME NOT NULL,
    TRANSACTION_REF VARCHAR(50) UNIQUE,

    FOREIGN KEY (BOOKING_ID) REFERENCES BOOKINGS(BOOKING_ID)
);

INSERT INTO AIRPORTS
(AIRPORT_CODE, AIRPORT_NAME, CITY, STATE, COUNTRY)
VALUES
('HYD', 'Rajiv Gandhi International Airport', 'Hyderabad', 'Telangana', 'India'),
('DEL', 'Indira Gandhi International Airport', 'Delhi', 'Delhi', 'India'),
('BOM', 'Chhatrapati Shivaji Maharaj Airport', 'Mumbai', 'Maharashtra', 'India'),
('MAA', 'Chennai International Airport', 'Chennai', 'Tamil Nadu', 'India'),
('BLR', 'Kempegowda International Airport', 'Bangalore', 'Karnataka', 'India'),
('VGA', 'Vijayawada International Airport', 'Vijayawada', 'Andhra Pradesh',
'India');

INSERT INTO AIRLINES
(AIRLINE_CODE, AIRLINE_NAME, COUNTRY)
VALUES
('6E', 'IndiGo', 'India'),
('AI', 'Air India', 'India'),
('UK', 'Vistara', 'India'),
('SG', 'SpiceJet', 'India'),
('EK', 'Emirates', 'UAE');

INSERT INTO AIRCRAFT
(AIRLINE_ID, AIRCRAFT_MODEL, REGISTRATION_NO, SEATING_CAPACITY)
VALUES
(1, 'Airbus A320', 'VT-IGA', 180),
```

```
(1, 'Airbus A321', 'VT-IGB', 220),
(1, 'ATR 72', 'VT-IGC', 78),
(2, 'Boeing 787', 'VT-AIA', 256),
(2, 'Airbus A320', 'VT-AIB', 180),
(3, 'Airbus A321', 'VT-VTA', 188),
(4, 'Boeing 737', 'VT-SGA', 189),
(5, 'Airbus A380', 'A6-EUA', 500);

INSERT INTO FLIGHTS
(FLIGHT_NO, AIRLINE_ID, AIRCRAFT_ID,
 DEPARTURE_AIRPORT_ID, ARRIVAL_AIRPORT_ID,
 DEPARTURE_DATETIME, ARRIVAL_DATETIME,
 ACTUAL_DEPARTURE, ACTUAL_ARRIVAL, FLIGHT_STATUS)
VALUES

('6E201', 1, 1, 1, 2,
 '2026-11-10 06:00:00', '2026-11-10 08:15:00',
 '2026-11-10 06:10:00', '2026-11-10 08:20:00', 'COMPLETED'),

('6E202', 1, 2, 2, 3,
 '2026-11-10 09:00:00', '2026-11-10 11:15:00',
 NULL, NULL, 'SCHEDULED'),

('AI301', 2, 4, 3, 1,
 '2026-11-10 12:00:00', '2026-11-10 13:30:00',
 '2026-11-10 12:05:00', '2026-11-10 13:35:00', 'COMPLETED'),

('UK401', 3, 6, 2, 5,
 '2026-11-11 07:00:00', '2026-11-11 09:30:00',
 NULL, NULL, 'SCHEDULED'),
```

('SG501', 4, 7, 4, 1,
'2026-11-11 10:00:00', '2026-11-11 11:30:00',
NULL, NULL, 'DELAYED'),

('6E203', 1, 3, 6, 1,
'2026-11-11 14:00:00', '2026-11-11 15:00:00',
NULL, NULL, 'SCHEDULED'),

('AI302', 2, 5, 1, 4,
'2026-11-12 08:00:00', '2026-11-12 10:00:00',
NULL, NULL, 'SCHEDULED'),

('EK601', 5, 8, 2, 3,
'2026-11-12 20:00:00', '2026-11-12 23:00:00',
NULL, NULL, 'SCHEDULED'),

('6E204', 1, 1, 5, 6,
'2026-11-13 06:30:00', '2026-11-13 08:00:00',
NULL, NULL, 'CANCELLED'),

('UK402', 3, 6, 5, 2,
'2026-11-13 11:00:00', '2026-11-13 13:30:00',
NULL, NULL, 'SCHEDULED'),

('SG502', 4, 7, 1, 3,
'2026-11-14 15:00:00', '2026-11-14 17:00:00',
NULL, NULL, 'SCHEDULED'),

('6E205', 1, 2, 3, 6,

'2026-11-14 18:00:00', '2026-11-14 20:00:00',

NULL, NULL, 'SCHEDULED');

INSERT INTO PASSENGERS

(PASSENGER_NAME, EMAIL, PHONE, PASSPORT_NO)

VALUES

('Rahul Sharma', 'rahul@gmail.com', '9876543210', 'P10001'),

('Priya Reddy', 'priya@gmail.com', '9876543211', 'P10002'),

('Arjun Kumar', 'arjun@gmail.com', '9876543212', 'P10003'),

('Sneha Rao', 'sneha@gmail.com', '9876543213', 'P10004'),

('Kiran Patel', 'kiran@gmail.com', '9876543214', 'P10005'),

('Ananya Singh', 'ananya@gmail.com', '9876543215', 'P10006'),

('Vikram Das', 'vikram@gmail.com', '9876543216', 'P10007'),

('Meera Nair', 'meera@gmail.com', '9876543217', 'P10008'),

('Rohit Verma', 'rohit@gmail.com', '9876543218', 'P10009'),

('Divya Rani', 'divya@gmail.com', '9876543219', 'P10010'),

('Sanjay Rao', 'sanjay@gmail.com', '9876543220', 'P10011'),

('Pooja Shah', 'pooja@gmail.com', '9876543221', 'P10012');

INSERT INTO BOOKINGS

(PASSENGER_ID, FLIGHT_ID, BOOKING_DATE,

SEAT_NO, TICKET_NO, FARE_AMOUNT, BOOKING_STATUS)

VALUES

(1, 1, '2026-11-01 10:00:00', '12A', 'TKT1001', 5500, 'CONFIRMED'),

(2, 1, '2026-11-02 11:00:00', '12B', 'TKT1002', 5500, 'CONFIRMED'),

(3, 2, '2026-11-02 12:00:00', '14A', 'TKT1003', 6200, 'CONFIRMED'),

(4, 2, '2026-11-03 09:00:00', '14B', 'TKT1004', 6200, 'PENDING'),

(5, 3, '2026-11-03 14:00:00', '10A', 'TKT1005', 4800, 'CONFIRMED'),

(6, 3, '2026-11-04 15:00:00', '10B', 'TKT1006', 4800, 'CONFIRMED'),

(7, 4, '2026-11-04 16:00:00', '15A', 'TKT1007', 7500, 'CONFIRMED'),

(8, 4, '2026-11-05 10:00:00', '15B', 'TKT1008', 7500, 'PENDING'),

(9, 5, '2026-11-05 11:00:00', '18A', 'TKT1009', 3900, 'CONFIRMED'),

(10, 5, '2026-11-06 12:00:00', '18B', 'TKT1010', 3900, 'CANCELLED'),

(11, 6, '2026-11-06 13:00:00', '08A', 'TKT1011', 2500, 'CONFIRMED'),

(12, 6, '2026-11-07 14:00:00', '08B', 'TKT1012', 2500, 'CONFIRMED'),

(1, 7, '2026-11-07 15:00:00', '20A', 'TKT1013', 5100, 'PENDING'),

(3, 8, '2026-11-07 16:00:00', '25A', 'TKT1014', 18000, 'CONFIRMED'),

(5, 9, '2026-11-08 17:00:00', '11A', 'TKT1015', 4200, 'CANCELLED'),

(7, 10, '2026-11-08 18:00:00', '16A', 'TKT1016', 6800, 'CONFIRMED'),

(9, 11, '2026-11-09 09:00:00', '19A', 'TKT1017', 5700, 'PENDING'),

(11, 12, '2026-11-09 10:00:00', '22A', 'TKT1018', 4400, 'CONFIRMED');

INSERT INTO PAYMENTS

(BOOKING_ID, AMOUNT, PAYMENT_METHOD,

PAYMENT_STATUS, PAYMENT_DATE, TRANSACTION_REF)

VALUES

(1, 5500, 'UPI', 'SUCCESS', '2026-11-01 10:05:00', 'TXN1001'),

(2, 5500, 'CARD', 'SUCCESS', '2026-11-02 11:05:00', 'TXN1002'),

(3, 6200, 'UPI', 'SUCCESS', '2026-11-02 12:05:00', 'TXN1003'),

(4, 6200, 'NETBANKING', 'FAILED', '2026-11-03 09:05:00', 'TXN1004'),

(5, 4800, 'CARD', 'SUCCESS', '2026-11-03 14:05:00', 'TXN1005'),

(6, 4800, 'UPI', 'SUCCESS', '2026-11-04 15:05:00', 'TXN1006'),

(7, 7500, 'CARD', 'SUCCESS', '2026-11-04 16:05:00', 'TXN1007'),

(8, 7500, 'UPI', 'PENDING', '2026-11-05 10:05:00', 'TXN1008'),

(9, 3900, 'UPI', 'SUCCESS', '2026-11-05 11:05:00', 'TXN1009'),

(10, 3900, 'CARD', 'REFUNDED', '2026-11-06 12:05:00', 'TXN1010'),

(11, 2500, 'UPI', 'SUCCESS', '2026-11-06 13:05:00', 'TXN1011'),

(12, 2500, 'CARD', 'SUCCESS', '2026-11-07 14:05:00', 'TXN1012'),

(13, 5100, 'UPI', 'FAILED', '2026-11-07 15:05:00', 'TXN1013'),

(14, 10000, 'CARD', 'SUCCESS', '2026-11-07 16:05:00', 'TXN1014'),

(14, 8000, 'UPI', 'SUCCESS', '2026-11-07 16:10:00', 'TXN1015'),

(15, 4200, 'UPI', 'REFUNDED', '2026-11-08 17:05:00', 'TXN1016'),

(16, 6800, 'CARD', 'SUCCESS', '2026-11-08 18:05:00', 'TXN1017');

PART 3: INDEXES

3.1 Introduction

Indexes are database objects used to improve data retrieval performance.

They help MySQL locate records efficiently when searching, joining, filtering or sorting data.

3.2 Index Implementation

```
CREATE INDEX IDX_AIRCRAFT_AIRLINE ON AIRCRAFT(AIRLINE_ID);

CREATE INDEX IDX_FLIGHTS_AIRLINE ON FLIGHTS(AIRLINE_ID);

CREATE INDEX IDX_FLIGHTS_AIRCRAFT ON FLIGHTS(AIRCRAFT_ID);

CREATE INDEX IDX_FLIGHTS_DEPARTURE ON
FLIGHTS(DEPARTURE_AIRPORT_ID);

CREATE INDEX IDX_FLIGHTS_ARRIVAL ON FLIGHTS(ARRIVAL_AIRPORT_ID);

CREATE INDEX IDX_FLIGHTS_DATE ON FLIGHTS(DEPARTURE_DATETIME);

CREATE INDEX IDX_BOOKINGS_PASSENGER ON
BOOKINGS(PASSENGER_ID);

CREATE INDEX IDX_BOOKINGS_FLIGHT ON BOOKINGS(FLIGHT_ID);
```

CREATE INDEX IDX_PAYMENTS_BOOKING ON PAYMENTS(BOOKING_ID);

Implementation section:

SHOW INDEX FROM AIRCRAFT;

SHOW INDEX FROM FLIGHTS;

SHOW INDEX FROM BOOKINGS;

SHOW INDEX FROM PAYMENTS;

PART 4: VIEWS

-- Q39. Create a view that displays complete flight details,

-- including airline, aircraft and airport information.

mysql> CREATE OR REPLACE VIEW VIEW_FLIGHT_DETAILS AS SELECT F.FLIGHT_ID, F.FLIGHT_NO, AL.AIRLINE_NAME, AC.AIRCRAFT_MODEL, DEP.AIRPORT_NAME AS DEPARTURE_AIRPORT, ARR.AIRPORT_NAME AS ARRIVAL_AIRPORT, F.DEPARTURE_DATETIME, F.ARRIVAL_DATETIME, F.FLIGHT_STATUS FROM FLIGHTS F JOIN AIRLINES AL ON F.AIRLINE_ID=AL.AIRLINE_ID JOIN AIRCRAFT AC ON F.AIRCRAFT_ID=AC.AIRCRAFT_ID JOIN AIRPORTS DEP ON F.DEPARTURE_AIRPORT_ID=DEP.AIRPORT_ID JOIN AIRPORTS ARR ON F.ARRIVAL_AIRPORT_ID=ARR.AIRPORT_ID;

mysql> SELECT * FROM VIEW_FLIGHT_DETAILS;

-- Q40. Create a view that displays booking statistics for every flight,

-- including flights without bookings.

mysql> CREATE OR REPLACE VIEW VIEW_FLIGHT_BOOKING_SUMMARY AS SELECT F.FLIGHT_ID, F.FLIGHT_NO, COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS, COALESCE(SUM(B.FARE_AMOUNT),0) AS TOTAL_BOOKING_FARE FROM FLIGHTS F LEFT JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID GROUP BY F.FLIGHT_ID, F.FLIGHT_NO;

mysql> SELECT * FROM VIEW_FLIGHT_BOOKING_SUMMARY;

-- Q41. Create a view that displays total successful payment revenue

-- generated by each airline.

mysql> CREATE OR REPLACE VIEW VIEW_AIRLINE_REVENUE AS SELECT AL.AIRLINE_ID, AL.AIRLINE_NAME, COALESCE(SUM(CASE WHEN P.PAYMENT_STATUS='SUCCESS' THEN P.AMOUNT ELSE 0 END),0) AS

```
TOTAL_REVENUE FROM AIRLINES AL LEFT JOIN FLIGHTS F ON
AL.AIRLINE_ID=F.AIRLINE_ID LEFT JOIN BOOKINGS B ON
F.FLIGHT_ID=B.FLIGHT_ID LEFT JOIN PAYMENTS P ON
B.BOOKING_ID=P.BOOKING_ID GROUP BY AL.AIRLINE_ID,
AL.AIRLINE_NAME;

mysql> SELECT * FROM VIEW_AIRLINE_REVENUE;


PART 5: STORED PROCEDURES

-- Q44. Create a stored procedure that searches available flights

-- using departure airport ID, arrival airport ID and travel date

-- as input parameters.


mysql> DELIMITER //


mysql> CREATE PROCEDURE SP_SEARCH_FLIGHTS(IN P_DEPARTURE_ID
INT, IN P_ARRIVAL_ID INT, IN P_TRAVEL_DATE DATE)

    -> BEGIN

    -> SELECT FLIGHT_ID, FLIGHT_NO, DEPARTURE_DATETIME,
ARRIVAL_DATETIME, FLIGHT_STATUS FROM FLIGHTS WHERE
DEPARTURE_AIRPORT_ID=P_DEPARTURE_ID AND
ARRIVAL_AIRPORT_ID=P_ARRIVAL_ID AND
DATE(DEPARTURE_DATETIME)=P_TRAVEL_DATE AND FLIGHT_STATUS IN
('SCHEDULED','DELAYED');

    -> END //


mysql> DELIMITER ;


-- Q45. Create a stored procedure to retrieve the complete

-- booking history of a passenger using Passenger ID.


mysql> DELIMITER //
```

mysql> CREATE PROCEDURE SP_PASSENGER_BOOKING_HISTORY(IN P_PASSENGER_ID INT)

-> BEGIN

-> SELECT P.PASSENGER_NAME, B.BOOKING_ID, F.FLIGHT_NO, B.BOOKING_DATE, B.SEAT_NO, B.FARE_AMOUNT, B.BOOKING_STATUS FROM PASSENGERS P JOIN BOOKINGS B ON P.PASSENGER_ID=B.PASSENGER_ID JOIN FLIGHTS F ON B.FLIGHT_ID=F.FLIGHT_ID WHERE P.PASSENGER_ID=P_PASSENGER_ID;

-> END //

mysql> DELIMITER ;

PART 6: TRIGGERS

-- Q46. Create a trigger that prevents a booking from being inserted

-- when the associated flight is cancelled or has already departed.

mysql> DELIMITER //

mysql> CREATE TRIGGER TRG_CHECK_FLIGHT_BEFORE_BOOKING

-> BEFORE INSERT ON BOOKINGS

-> FOR EACH ROW

-> BEGIN

-> DECLARE V_STATUS VARCHAR(20);

-> DECLARE V_DEPARTURE DATETIME;

-> SELECT FLIGHT_STATUS, DEPARTURE_DATETIME

-> INTO V_STATUS, V_DEPARTURE

-> FROM FLIGHTS

-> WHERE FLIGHT_ID=NEW.FLIGHT_ID;

-> IF V_STATUS='CANCELLED' OR V_DEPARTURE<=NOW() THEN

-> SIGNAL SQLSTATE '45000'

-> SET MESSAGE_TEXT='Cannot book a cancelled or departed flight';

```
    -> END IF;
    -> END //
mysql> DELIMITER ;
-- Q47. Create a trigger that automatically confirms a booking
-- after the required successful payment amount has been recorded.
mysql> DELIMITER //
mysql> CREATE TRIGGER TRG_CONFIRM_BOOKING_AFTER_PAYMENT
    -> AFTER INSERT ON PAYMENTS
    -> FOR EACH ROW
    -> BEGIN
    -> DECLARE V_PAID DECIMAL(10,2);
    -> DECLARE V_FARE DECIMAL(10,2);
    -> IF NEW.PAYMENT_STATUS='SUCCESS' THEN
    -> SELECT FARE_AMOUNT INTO V_FARE
    -> FROM BOOKINGS
    -> WHERE BOOKING_ID=NEW.BOOKING_ID;
    -> SELECT COALESCE(SUM(AMOUNT),0) INTO V_PAID
    -> FROM PAYMENTS
    -> WHERE BOOKING_ID=NEW.BOOKING_ID
    -> AND PAYMENT_STATUS='SUCCESS';
    -> IF V_PAID>=V_FARE THEN
    -> UPDATE BOOKINGS
    -> SET BOOKING_STATUS='CONFIRMED'
    -> WHERE BOOKING_ID=NEW.BOOKING_ID;
    -> END IF;
    -> END IF;
    -> END //
mysql> DELIMITER ;
```

PART 7: QUERY BANK – LEVEL 1: SIMPLE QUERIES (Q1–Q6)

Concept focus: SELECT, WHERE, ORDER BY, DISTINCT, comparison operators and basic filtering.

Q1. Retrieve all airport names, airport codes and cities from the AIRPORTS table.
mysql> SELECT AIRPORT_NAME,AIRPORT_CODE,CITY FROM AIRPORTS;

+----------+----------+----------+

| AIRPORT_NAME | AIRPORT_CODE | CITY |

+----------+----------+----------+

| Rajiv Gandhi International Airport | HYD | Hyderabad |

| Indira Gandhi International Airport | DEL | Delhi |

| Chhatrapati Shivaji Maharaj Airport | BOM | Mumbai |

| Chennai International Airport | MAA | Chennai |

| Kempegowda International Airport | BLR | Bangalore |

| Vijayawada International Airport | VGA | Vijayawada |

+----------+----------+----------+

Q2. Display all flights whose status is DELAYED.
SELECT*FROM FLIGHTS WHERE FLIGHT_STATUS='DELAYED';

+----------+----------+----------+----------+----------+----------+----------+----------+----------+----------+

| FLIGHT_ID | FLIGHT_NO | AIRLINE_ID | AIRCRAFT_ID | DEPARTURE_AIRPORT_ID | ARRIVAL_AIRPORT_ID | DEPARTURE_DATETIME | ARRIVAL_DATETIME | ACTUAL_DEPARTURE | ACTUAL_ARRIVAL | FLIGHT_STATUS |

+----------+----------+----------+----------+----------+----------+----------+----------+----------+----------+

| 5 | SG501 | 4 | 7 | 4 | 1 | 2026-11-11 10:00:00 | 2026-11-11 11:30:00 | NULL | NULL | DELAYED |

+----------+----------+----------+----------+----------+----------+----------+----------+----------+----------+

Q3. Retrieve all passengers whose names begin with the letter 'A'.
mysql> SELECT PASSENGER_NAME FROM PASSENGERS WHERE PASSENGER_NAME LIKE 'A%';

+----------+

| PASSENGER_NAME |

+---------------+

| Arjun Kumar |

| Ananya Singh |

+---------------+

2 rows in set (0.00 sec)

Q4. Display all flights scheduled between two specified dates.
SELECT*FROM FLIGHTS WHERE FLIGHT_STATUS = 'SCHEDULED';

Q5. Retrieve bookings whose fare amount is greater than ₹5,000 and less than ₹15,000.
mysql> SELECT *FROM BOOKINGS WHERE FARE_AMOUNT > 5000 AND FARE_AMOUNT < 15000;

Q6. Display all passengers whose email address or phone number is missing.

mysql> SELECT PASSENGER_NAME FROM PASSENGERS WHERE EMAIL IS NULL OR PHONE IS NULL;

PART 8: QUERY BANK – LEVEL 2: INTERMEDIATE QUERIES (Q7–Q14)

Concept focus: INSERT, UPDATE, DELETE, arithmetic operators, logical operators, aggregate functions and GROUP BY.

Q7. Insert a new airport record with all required airport details.
mysql> INSERT INTO AIRPORTS (AIRPORT_CODE, AIRPORT_NAME, CITY, STATE, COUNTRY) VALUES ('IXE', 'Mangalore International Airport', 'Mangalore', 'Karnataka', 'India');

Q8. Increase the fare amount of confirmed bookings belonging to a specified airline by 10%.

mysql> UPDATE BOOKINGS SET FARE_AMOUNT = FARE_AMOUNT*1.1 WHERE BOOKING_STATUS='CONFIRMED';

Query OK, 12 rows affected (0.01 sec)

Q9. Update the status of pending payments older than a specified date.
UPDATE PAYMENTS SET PAYMENT_STATUS = 'FAILED' WHERE PAYMENT_STATUS = 'PENDING' AND PAYMENT_DATE < '2026-11-05';

Q10. Find the total number of flights operated by each airline.
SELECT AIRLINE_ID, COUNT(FLIGHT_ID) AS TOTAL_FLIGHTS FROM FLIGHTS

GROUP BY AIRLINE_ID;

Q11. Calculate the average, minimum and maximum fare amount of all bookings.
SELECT AVG(FARE_AMOUNT) AS AVERAGE_FARE,MIN(FARE_AMOUNT) AS MINIMUM_FARE,MAX(FARE_AMOUNT) AS MAXIMUM_FARE FROM BOOKINGS;

Q12. Find the number of passengers booked on each flight.

SELECT FLIGHT_ID,COUNT(PASSENGER_ID) AS TOTAL_PASSENGERS FROM BOOKINGS

GROUP BY FLIGHT_ID;

Q13. Display airlines that own more than three aircraft.
SELECT AIRLINE_ID,COUNT(AIRCRAFT_ID) AS TOTAL_AIRCRAFT FROM AIRCRAFT GROUP BY AIRLINE_ID HAVING COUNT(AIRCRAFT_ID) > 3;

Q14. Calculate the total successful payment amount collected through each payment method.
SELECT PAYMENT_METHOD,SUM(AMOUNT) AS TOTAL_AMOUNT FROM PAYMENTS

WHERE PAYMENT_STATUS = 'SUCCESS' GROUP BY PAYMENT_METHOD;

PART 9: QUERY BANK – LEVEL 3: ADVANCED QUERIES (Q15–Q23)

Concept focus: HAVING, joins, LEFT JOIN, NULL handling, date functions and conditional expressions.

Q15. Find the average booking fare for each flight and display only flights whose average fare is greater than ₹8,000.
mysql> SELECT FLIGHT_ID, AVG(FARE_AMOUNT) AS AVG_FARE FROM BOOKINGS GROUP BY FLIGHT_ID HAVING AVG(FARE_AMOUNT)>8000;

Q16. Count the number of bookings made by each passenger, including passengers who have never made a booking.
mysql> SELECT P.PASSENGER_ID, P.PASSENGER_NAME, (SELECT COUNT(*) FROM BOOKINGS B WHERE B.PASSENGER_ID=P.PASSENGER_ID) AS TOTAL_BOOKINGS FROM PASSENGERS P;

Q17. Display flight number, airline name, aircraft model, departure airport and arrival airport for every flight.
mysql> SELECT F.FLIGHT_NO, AL.AIRLINE_NAME, AC.AIRCRAFT_MODEL, DEP.AIRPORT_NAME AS DEPARTURE_AIRPORT, ARR.AIRPORT_NAME AS ARRIVAL_AIRPORT FROM FLIGHTS F JOIN AIRLINES AL ON F.AIRLINE_ID=AL.AIRLINE_ID JOIN AIRCRAFT AC ON F.AIRCRAFT_ID=AC.AIRCRAFT_ID JOIN AIRPORTS DEP ON

F.DEPARTURE_AIRPORT_ID=DEP.AIRPORT_ID JOIN AIRPORTS ARR ON F.ARRIVAL_AIRPORT_ID=ARR.AIRPORT_ID;

Q18. Retrieve passenger names, booking IDs, flight numbers and booking statuses for all confirmed bookings.
mysql> SELECT P.PASSENGER_NAME, B.BOOKING_ID, F.FLIGHT_NO, B.BOOKING_STATUS FROM BOOKINGS B JOIN PASSENGERS P ON B.PASSENGER_ID=P.PASSENGER_ID JOIN FLIGHTS F ON B.FLIGHT_ID=F.FLIGHT_ID WHERE B.BOOKING_STATUS='CONFIRMED';

Q19. Display all airlines, including airlines that have not registered any aircraft.
mysql> SELECT AL.AIRLINE_ID, AL.AIRLINE_NAME, AC.AIRCRAFT_ID, AC.AIRCRAFT_MODEL FROM AIRLINES AL LEFT JOIN AIRCRAFT AC ON AL.AIRLINE_ID=AC.AIRLINE_ID;

Q20. Find aircraft that have never been assigned to any flight.
mysql> SELECT * FROM AIRCRAFT AC WHERE NOT EXISTS (SELECT 1 FROM FLIGHTS F WHERE F.AIRCRAFT_ID=AC.AIRCRAFT_ID);

Q21. Display all passengers with their booking details, including passengers who have never made a booking.
mysql> SELECT P.PASSENGER_NAME, B.BOOKING_ID, B.FLIGHT_ID, B.BOOKING_DATE, B.BOOKING_STATUS FROM PASSENGERS P LEFT JOIN BOOKINGS B ON P.PASSENGER_ID=B.PASSENGER_ID;

Q22. Retrieve all bookings and their payment details, including bookings without payment records.
mysql> SELECT B.BOOKING_ID, B.FARE_AMOUNT, P.PAYMENT_ID, P.AMOUNT, P.PAYMENT_METHOD, P.PAYMENT_STATUS FROM BOOKINGS B LEFT JOIN PAYMENTS P ON B.BOOKING_ID=P.BOOKING_ID;

Q23. Generate a flight operations report showing scheduled departure, actual departure and departure delay information where actual departure is available.
mysql> SELECT FLIGHT_NO, DEPARTURE_DATETIME AS SCHEDULED_DEPARTURE, ACTUAL_DEPARTURE, TIMESTAMPDIFF(MINUTE,DEPARTURE_DATETIME,ACTUAL_DEPARTURE) AS DELAY_MINUTES FROM FLIGHTS WHERE ACTUAL_DEPARTURE IS NOT NULL;

PART 10: QUERY BANK – LEVEL 4: COMPLEX QUERIES (Q24–Q33)

Concept focus: Subqueries, correlated subqueries, EXISTS, NOT EXISTS, nested queries and advanced grouping.

Q24. Find passengers who have never made a booking using a subquery.
mysql> SELECT PASSENGER_ID, PASSENGER_NAME FROM PASSENGERS WHERE PASSENGER_ID NOT IN (SELECT PASSENGER_ID FROM BOOKINGS);

Q25. Retrieve flights whose booking count is greater than the average booking count across all flights.
mysql> SELECT FLIGHT_ID, COUNT(*) AS TOTAL_BOOKINGS FROM BOOKINGS GROUP BY FLIGHT_ID HAVING COUNT(*)>(SELECT AVG(BOOKING_COUNT) FROM (SELECT COUNT(*) AS BOOKING_COUNT FROM BOOKINGS GROUP BY FLIGHT_ID) AS T);

Q26. Find bookings whose fare amount is greater than the average fare amount of all bookings.
mysql> SELECT * FROM BOOKINGS WHERE FARE_AMOUNT>(SELECT AVG(FARE_AMOUNT) FROM BOOKINGS);

Q27. Find the second-highest distinct booking fare in the database.
mysql> SELECT MAX(FARE_AMOUNT) AS SECOND_HIGHEST_FARE FROM BOOKINGS WHERE FARE_AMOUNT<(SELECT MAX(FARE_AMOUNT) FROM BOOKINGS);

Q28. Retrieve airlines that operate flights having at least one successful payment.
mysql> SELECT AIRLINE_ID, AIRLINE_NAME FROM AIRLINES AL WHERE EXISTS (SELECT 1 FROM FLIGHTS F JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID JOIN PAYMENTS P ON B.BOOKING_ID=P.BOOKING_ID WHERE F.AIRLINE_ID=AL.AIRLINE_ID AND P.PAYMENT_STATUS='SUCCESS');

Q29. Find flights that have no confirmed bookings using NOT EXISTS.
mysql> SELECT FLIGHT_ID, FLIGHT_NO FROM FLIGHTS F WHERE NOT EXISTS (SELECT 1 FROM BOOKINGS B WHERE B.FLIGHT_ID=F.FLIGHT_ID AND B.BOOKING_STATUS='CONFIRMED');

Q30. Find passengers who have booked flights operated by more than one airline.
mysql> SELECT P.PASSENGER_ID, P.PASSENGER_NAME FROM PASSENGERS P JOIN BOOKINGS B ON P.PASSENGER_ID=B.PASSENGER_ID JOIN FLIGHTS F ON B.FLIGHT_ID=F.FLIGHT_ID GROUP BY P.PASSENGER_ID, P.PASSENGER_NAME HAVING COUNT(DISTINCT F.AIRLINE_ID)>1;

Q31. Retrieve flights whose successful payment revenue is greater than the average successful payment revenue of all flights.
mysql> SELECT F.FLIGHT_ID, F.FLIGHT_NO, SUM(P.AMOUNT) AS TOTAL_REVENUE FROM FLIGHTS F JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID JOIN PAYMENTS P ON B.BOOKING_ID=P.BOOKING_ID WHERE P.PAYMENT_STATUS='SUCCESS' GROUP BY F.FLIGHT_ID, F.FLIGHT_NO HAVING SUM(P.AMOUNT)>(SELECT

AVG(FLIGHT_REVENUE) FROM (SELECT SUM(PM.AMOUNT) AS FLIGHT_REVENUE FROM FLIGHTS FL JOIN BOOKINGS BK ON FL.FLIGHT_ID=BK.FLIGHT_ID JOIN PAYMENTS PM ON BK.BOOKING_ID=PM.BOOKING_ID WHERE PM.PAYMENT_STATUS='SUCCESS' GROUP BY FL.FLIGHT_ID) AS T);

Q32. Find the airline with the highest number of confirmed bookings.
mysql> SELECT AL.AIRLINE_ID, AL.AIRLINE_NAME, COUNT(B.BOOKING_ID) AS CONFIRMED_BOOKINGS FROM AIRLINES AL JOIN FLIGHTS F ON AL.AIRLINE_ID=F.AIRLINE_ID JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID WHERE B.BOOKING_STATUS='CONFIRMED' GROUP BY AL.AIRLINE_ID, AL.AIRLINE_NAME ORDER BY CONFIRMED_BOOKINGS DESC LIMIT 1;

Q33. Display each airline's total flight count, confirmed booking count and successful payment revenue.
mysql> SELECT AL.AIRLINE_ID, AL.AIRLINE_NAME, (SELECT COUNT(*) FROM FLIGHTS F WHERE F.AIRLINE_ID=AL.AIRLINE_ID) AS TOTAL_FLIGHTS, (SELECT COUNT(*) FROM FLIGHTS F JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID WHERE F.AIRLINE_ID=AL.AIRLINE_ID AND B.BOOKING_STATUS='CONFIRMED') AS CONFIRMED_BOOKINGS, (SELECT COALESCE(SUM(P.AMOUNT),0) FROM FLIGHTS F JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID JOIN PAYMENTS P ON B.BOOKING_ID=P.BOOKING_ID WHERE F.AIRLINE_ID=AL.AIRLINE_ID AND P.PAYMENT_STATUS='SUCCESS') AS SUCCESSFUL_REVENUE FROM AIRLINES AL;

PART 11: QUERY BANK – LEVEL 5: REAL-WORLD AND ADVANCED SQL (Q34–Q43)

Concept focus: CASE, UNION, CTEs, window functions, views, indexes and query optimization.

Q34. Generate a booking report that classifies each booking as PAID, PARTIALLY_PAID, UNPAID or CANCELLED based on its booking status and payment information.
mysql> SELECT B.BOOKING_ID, B.BOOKING_STATUS, B.FARE_AMOUNT, COALESCE(SUM(CASE WHEN P.PAYMENT_STATUS='SUCCESS' THEN P.AMOUNT ELSE 0 END),0) AS PAID_AMOUNT, CASE WHEN B.BOOKING_STATUS='CANCELLED' THEN 'CANCELLED' WHEN COALESCE(SUM(CASE WHEN P.PAYMENT_STATUS='SUCCESS' THEN P.AMOUNT ELSE 0 END),0)=0 THEN 'UNPAID' WHEN COALESCE(SUM(CASE WHEN P.PAYMENT_STATUS='SUCCESS' THEN P.AMOUNT ELSE 0 END),0)<B.FARE_AMOUNT THEN 'PARTIALLY_PAID' ELSE 'PAID' END AS PAYMENT_CATEGORY FROM BOOKINGS B LEFT JOIN PAYMENTS P ON B.BOOKING_ID=P.BOOKING_ID GROUP BY B.BOOKING_ID, B.BOOKING_STATUS, B.FARE_AMOUNT;

Q35. Display a combined list of departure and arrival airport codes for all flights using UNION.

```
mysql> SELECT DISTINCT A.AIRPORT_CODE FROM FLIGHTS F JOIN AIRPORTS A ON F.DEPARTURE_AIRPORT_ID=A.AIRPORT_ID UNION SELECT DISTINCT A.AIRPORT_CODE FROM FLIGHTS F JOIN AIRPORTS A ON F.ARRIVAL_AIRPORT_ID=A.AIRPORT_ID;
```

Q36. Use a Common Table Expression (CTE) to identify flights with the highest booking counts.

```
mysql> WITH FLIGHT_COUNTS AS (SELECT FLIGHT_ID, COUNT(*) AS TOTAL_BOOKINGS FROM BOOKINGS GROUP BY FLIGHT_ID) SELECT FLIGHT_ID, TOTAL_BOOKINGS FROM FLIGHT_COUNTS WHERE TOTAL_BOOKINGS=(SELECT MAX(TOTAL_BOOKINGS) FROM FLIGHT_COUNTS);
```

Q37. Rank flights according to the total number of confirmed bookings using a window function.

```
mysql> SELECT FLIGHT_ID, TOTAL_BOOKINGS, RANK() OVER (ORDER BY TOTAL_BOOKINGS DESC) AS FLIGHT_RANK FROM (SELECT F.FLIGHT_ID, COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS FROM FLIGHTS F LEFT JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID AND B.BOOKING_STATUS='CONFIRMED' GROUP BY F.FLIGHT_ID) AS T;
```

Q38. Find the top three passengers based on total successful payment amounts associated with their bookings.

```
mysql> SELECT P.PASSENGER_ID, P.PASSENGER_NAME, SUM(PM.AMOUNT) AS TOTAL_PAID FROM PASSENGERS P JOIN BOOKINGS B ON P.PASSENGER_ID=B.PASSENGER_ID JOIN PAYMENTS PM ON B.BOOKING_ID=PM.BOOKING_ID WHERE PM.PAYMENT_STATUS='SUCCESS' GROUP BY P.PASSENGER_ID, P.PASSENGER_NAME ORDER BY TOTAL_PAID DESC LIMIT 3;
```

Q39. Create a view that displays complete flight details, including airline, aircraft and airport information.

```
mysql> CREATE VIEW VIEW_FLIGHT_DETAILS AS SELECT F.FLIGHT_ID, F.FLIGHT_NO, AL.AIRLINE_NAME, AC.AIRCRAFT_MODEL, DEP.AIRPORT_NAME AS DEPARTURE_AIRPORT, ARR.AIRPORT_NAME AS ARRIVAL_AIRPORT, F.DEPARTURE_DATETIME, F.ARRIVAL_DATETIME, F.FLIGHT_STATUS FROM FLIGHTS F JOIN AIRLINES AL ON F.AIRLINE_ID=AL.AIRLINE_ID JOIN AIRCRAFT AC ON F.AIRCRAFT_ID=AC.AIRCRAFT_ID JOIN AIRPORTS DEP ON F.DEPARTURE_AIRPORT_ID=DEP.AIRPORT_ID JOIN AIRPORTS ARR ON F.ARRIVAL_AIRPORT_ID=ARR.AIRPORT_ID;
```

Q40. Create a view that displays booking statistics for every flight, including flights without bookings.

mysql> CREATE VIEW VIEW_FLIGHT_BOOKING_SUMMARY AS SELECT F.FLIGHT_ID, F.FLIGHT_NO, COUNT(B.BOOKING_ID) AS TOTAL_BOOKINGS, COALESCE(SUM(B.FARE_AMOUNT),0) AS TOTAL_BOOKING_FARE FROM FLIGHTS F LEFT JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID GROUP BY F.FLIGHT_ID, F.FLIGHT_NO;

Q41. Create a view that displays total successful payment revenue generated by each airline.
mysql> CREATE VIEW VIEW_AIRLINE_REVENUE AS SELECT AL.AIRLINE_ID, AL.AIRLINE_NAME, COALESCE(SUM(CASE WHEN P.PAYMENT_STATUS='SUCCESS' THEN P.AMOUNT ELSE 0 END),0) AS TOTAL_REVENUE FROM AIRLINES AL LEFT JOIN FLIGHTS F ON AL.AIRLINE_ID=F.AIRLINE_ID LEFT JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID LEFT JOIN PAYMENTS P ON B.BOOKING_ID=P.BOOKING_ID GROUP BY AL.AIRLINE_ID, AL.AIRLINE_NAME;

Q42. Create indexes on frequently searched flight and booking columns.
mysql> CREATE INDEX IDX_FLIGHTS_AIRLINE ON FLIGHTS(AIRLINE_ID);
mysql> CREATE INDEX IDX_FLIGHTS_DEPARTURE ON FLIGHTS(DEPARTURE_AIRPORT_ID);
mysql> CREATE INDEX IDX_FLIGHTS_ARRIVAL ON FLIGHTS(ARRIVAL_AIRPORT_ID);
mysql> CREATE INDEX IDX_FLIGHTS_DATE ON FLIGHTS(DEPARTURE_DATETIME);
mysql> CREATE INDEX IDX_BOOKINGS_PASSENGER ON BOOKINGS(PASSENGER_ID);
mysql> CREATE INDEX IDX_BOOKINGS_FLIGHT ON BOOKINGS(FLIGHT_ID);

Q43. Use EXPLAIN to analyze a query involving flight searching and booking information, and compare its execution plan before and after indexing.
mysql> EXPLAIN SELECT F.FLIGHT_ID, F.FLIGHT_NO, B.BOOKING_ID FROM FLIGHTS F JOIN BOOKINGS B ON F.FLIGHT_ID=B.FLIGHT_ID WHERE F.DEPARTURE_AIRPORT_ID=4 AND F.ARRIVAL_AIRPORT_ID=1 AND F.DEPARTURE_DATETIME>='2026-11-10' AND F.DEPARTURE_DATETIME<'2026-11-15';

PART 12: QUERY BANK – LEVEL 6: DATABASE PROGRAMMING (Q44–Q47)

Concept focus: Stored procedures, triggers, transactions, error handling and business-rule enforcement.

Q44. Create a stored procedure that searches available flights using departure airport ID, arrival airport ID and travel date as input parameters.

```
mysql> DELIMITER //

mysql> CREATE PROCEDURE SP_SEARCH_FLIGHTS(IN P_DEPARTURE_ID INT, IN P_ARRIVAL_ID INT, IN P_TRAVEL_DATE DATE)

    -> BEGIN

    -> SELECT FLIGHT_ID, FLIGHT_NO, DEPARTURE_DATETIME, ARRIVAL_DATETIME, FLIGHT_STATUS FROM FLIGHTS WHERE DEPARTURE_AIRPORT_ID=P_DEPARTURE_ID AND ARRIVAL_AIRPORT_ID=P_ARRIVAL_ID AND DATE(DEPARTURE_DATETIME)=P_TRAVEL_DATE AND FLIGHT_STATUS IN ('SCHEDULED','DELAYED');

    -> END //

mysql> DELIMITER ;
mysql> CALL SP_SEARCH_FLIGHTS(4,1,'2026-11-11');
```

Q45. Create a trigger that prevents a booking from being inserted when the associated flight is cancelled or has already departed.

```
mysql> DELIMITER //

mysql> CREATE TRIGGER TRG_CHECK_FLIGHT_BEFORE_BOOKING BEFORE INSERT ON BOOKINGS FOR EACH ROW

    -> BEGIN

    -> DECLARE V_STATUS VARCHAR(20);

    -> DECLARE V_DEPARTURE DATETIME;

    -> SELECT FLIGHT_STATUS, DEPARTURE_DATETIME INTO V_STATUS, V_DEPARTURE FROM FLIGHTS WHERE FLIGHT_ID=NEW.FLIGHT_ID;

    -> IF V_STATUS='CANCELLED' OR V_DEPARTURE<=NOW() THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Cannot book a cancelled or departed flight';

    -> END IF;

    -> END //

mysql> DELIMITER ;
```

Q46. Create a trigger that automatically confirms a booking after the required successful payment amount has been recorded.

```
mysql> DELIMITER //
```

```
mysql> CREATE TRIGGER TRG_CONFIRM_BOOKING_AFTER_PAYMENT
AFTER INSERT ON PAYMENTS FOR EACH ROW
    -> BEGIN
    -> DECLARE V_PAID DECIMAL(10,2);
    -> DECLARE V_FARE DECIMAL(10,2);
    -> IF NEW.PAYMENT_STATUS='SUCCESS' THEN
    -> SELECT FARE_AMOUNT INTO V_FARE FROM BOOKINGS WHERE
BOOKING_ID=NEW.BOOKING_ID;
    -> SELECT COALESCE(SUM(AMOUNT),0) INTO V_PAID FROM PAYMENTS
WHERE BOOKING_ID=NEW.BOOKING_ID AND PAYMENT_STATUS='SUCCESS';
    -> IF V_PAID>=V_FARE THEN
    -> UPDATE BOOKINGS SET BOOKING_STATUS='CONFIRMED' WHERE
BOOKING_ID=NEW.BOOKING_ID;
    -> END IF;
    -> END IF;
    -> END //
mysql> DELIMITER ;
```

PART 15: PROJECT CONCLUSION

The Airport Operations & Flight Management System demonstrates the practical implementation of relational database concepts using MySQL.

The project covers database design, entity relationships, normalization, table creation, constraints, sample data, indexing, views, stored procedures, triggers, joins, subqueries, aggregate functions, CTEs, window functions and transaction management.

It provides a structured database model for managing flight operations, passenger reservations and payment transactions.

The project demonstrates how SQL concepts can be applied to realistic business scenarios while maintaining data integrity, reducing redundancy and supporting efficient information retrieval.